BCS703: CNS MODULE 2

QUESTION 1:

Explain the Diffie–Hellman Key Exchange Algorithm with an Example and Diagram

1. Introduction

The **Diffie–Hellman key exchange** is a public-key cryptographic technique used by two users to **securely establish a common secret key** over an insecure communication channel.

The algorithm itself is used for **exchange of secret values**. The resulting shared secret is typically used as a **symmetric key** for subsequent encryption of messages. Its security is based on the computational difficulty of the **discrete logarithm problem**.

2. Public and Private Values

The algorithm uses two publicly known values:

- q = a large **prime number**
- a = a **primitive root of** q

Two users, **A (Alice)** and **B (Bob)**, independently choose private keys.

Let:

- X_A = private key of A
- Y_A = public key of A
- X_B = private key of B
- Y_B = public key of B

The public keys are exchanged, but the private keys are never transmitted. BCS703-module-2-textbook

---

3. Diffie–Hellman Algorithm

Step 1: Public parameters

Alice and Bob agree on:

$$q = \text{prime number}$$

$$a = \text{primitive root of } q$$

These values can be known publicly.

Step 2: Alice generates her keys

Alice selects a private key:

$$X_A < q$$

She calculates her public key:

$$\boxed{Y_A = a^{X_A} \pmod q}$$

Alice sends Y_A to Bob.

Step 3: Bob generates his keys

Bob selects a private key:

$$X_B < q$$

He calculates his public key:

$$\boxed{Y_B = a^{X_B} \pmod q}$$

Bob sends Y_B to Alice.

Step 4: Alice calculates the shared secret

After receiving Bob's public key:

$$\boxed{K = (Y_B)^{X_A} \pmod q}$$

Step 5: Bob calculates the shared secret

After receiving Alice's public key:

$$\boxed{K = (Y_A)^{X_B} \pmod q}$$

Both calculations produce the **same secret key.** BCS703-module-2-textbook

4. Why Both Users Obtain the Same Key

Alice calculates:

$$K = (Y_B)^{X_A} \pmod{q}$$

Since

$$Y_B = a^{X_B} \pmod{q}$$

therefore,

$$K = (a^{X_B})^{X_A} \pmod{q}$$

$$K = a^{X_B X_A} \pmod{q}$$

Similarly, Bob calculates:

$$K = (Y_A)^{X_B} \pmod{q}$$

Since

$$Y_A = a^{X_A} \pmod{q}$$

we get:

$$K = (a^{X_A})^{X_B} \pmod{q}$$

$$K = a^{X_A X_B} \pmod{q}$$

Since

$$X_A X_B = X_B X_A$$

both users obtain:

$$\boxed{K = a^{X_A X_B} \pmod{q}}$$

Thus, Alice and Bob have established a common secret key without transmitting the secret key itself.

5. Example

Consider the following values:

$$q = 353$$

$$a = 3$$

where 3 is a primitive root of 353.

Let Alice select:

$$X_A = 97$$

and Bob select:

$$X_B = 233$$

The private keys are kept secret.

Alice calculates her public key

$$Y_A = 3^{97} \pmod{353}$$

$$\boxed{Y_A = 40}$$

Bob calculates his public key

$$Y_B = 3^{233} \pmod{353}$$

$$\boxed{Y_B = 248}$$

Alice and Bob exchange these public values.

6. Calculation of Common Secret Key

Alice's calculation

Alice receives $Y_B = 248$.

$$K = (Y_B)^{X_A} \pmod{353}$$

$$K = 248^{97} \pmod{353}$$

$$\boxed{K = 160}$$

Bob's calculation

Bob receives $Y_A = 40$.

$$K = (Y_A)^{X_B} \pmod{353}$$

$$K = 40^{233} \pmod{353}$$

$$\boxed{K = 160}$$

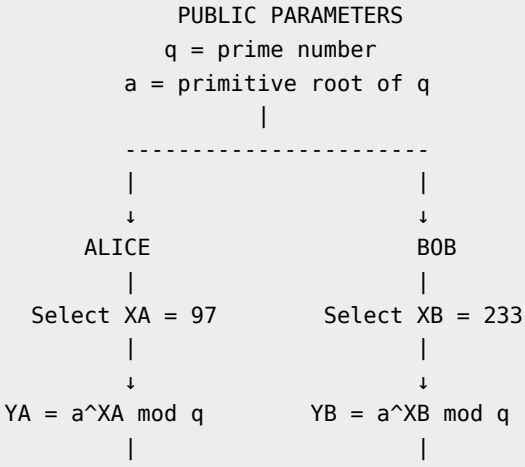
Therefore,

$$\boxed{\text{Shared Secret Key} = 160}$$

The example values and calculations correspond to the standard Diffie–Hellman example given in the reference material. BCS703-module-2-textbook

7. Diagram

Draw the following diagram in the examination:



```
       YA = 40              YB = 248
          |                     |
          |------ Exchange -----|
          |                     |
          ↓                     ↓
K = YB^XA mod q         K = YA^XB mod q
          |                     |
       K = 160              K = 160
          |                     |
          --------- ------------
                    ↓
             SHARED SECRET KEY
                    K = 160
```

8. Security of Diffie–Hellman

An attacker can observe the public values:

$$q, \quad a, \quad Y_A, \quad Y_B$$

However, the private values X_A and X_B are not transmitted.

To obtain a private key from a public key, an attacker would need to solve a **discrete logarithm** problem, such as:

$$X_B = \log_a(Y_B) \pmod q$$

For sufficiently large values, computing the discrete logarithm is computationally infeasible. This difficulty provides the security basis of the Diffie–Hellman key exchange. BCS703-module-2-textbook

Question 2:

Perform encryption and decryption using the RSA algorithm given public key e=6 for two prime numbers 17 and 31 with message 3.

Answer

Given

$$p = 17, \qquad q = 31$$

Public exponent:

$$e = 6$$

Message:

$$M = 3$$

For RSA:

$$n = pq$$

$$\phi(n) = (p-1)(q-1)$$

Encryption:

$$C = M^e \bmod n$$

Decryption requires a private key d satisfying:

$$ed \equiv 1 \pmod{\phi(n)}$$

---

1. Calculate n

$$n = pq$$

$$n = 17 \times 31$$

$$\boxed{n = 527}$$

2. Calculate Euler's Totient Function

$$\phi(n) = (p-1)(q-1)$$

$$\phi(527) = (17-1)(31-1)$$

$$= 16 \times 30$$

$$\boxed{\phi(n) = 480}$$

---

3. Check the Public Key $e = 6$

For RSA, e must be relatively prime to $\phi(n)$:

$$\gcd(e, \phi(n)) = 1$$

Here,

$$\gcd(6, 480) = 6$$

Since

$$\boxed{\gcd(6, 480) \neq 1}$$

the given value $e = 6$ is **not a valid RSA public exponent**.

Therefore, a valid private key d cannot be calculated.

---

4. Encryption

Although the given $e = 6$ is not a valid RSA public exponent, the encryption expression can still be evaluated for the given message.

$$C = M^e \bmod n$$

$$C = 3^6 \bmod 527$$

$$3^6 = 729$$

Therefore,

$$C = 729 \bmod 527$$

$$\boxed{C = 202}$$

Hence, the ciphertext obtained is:

$$\boxed{C = 202}$$

---

5. Decryption

For RSA decryption, we need:

$$d = e^{-1} \pmod{\phi(n)}$$

Therefore:

$$6d \equiv 1 \pmod{480}$$

This equation has **no solution**, because 6 and 480 are not relatively prime.

Thus:

$$\boxed{\text{Private key } d \text{ does not exist}}$$

and consequently the ciphertext 202 **cannot be validly decrypted using RSA with the given public exponent** $e = 6$.

Final Answer

$$\boxed{p = 17, \quad q = 31}$$

$$\boxed{n = 527}$$

$$\boxed{\phi(n) = 480}$$

$$\boxed{e = 6}$$

Since:

$$\boxed{\gcd(6, 480) = 6 \neq 1}$$

the given public key is **invalid for RSA**.

Encryption:

$$\boxed{C = 3^6 \bmod 527 = 202}$$

Decryption:

$$\boxed{\text{Not possible because } d = 6^{-1} \pmod{480} \text{ does not exist}}$$

Important examination point

The numerical question as shown contains a **mathematical inconsistency**: $e = 6$ cannot be an RSA public exponent for $p = 17,\ q = 31$. Do **not** fabricate a value of d. The correct solution is to identify that $e = 6$ is invalid. The standard RSA requirement is that e be relatively prime to $\phi(n)$. BCS703-module-2-textbook

QUESTION 3:

Briefly Explain the Security Aspects of the RSA Algorithm

Introduction

RSA is a **public-key cryptographic algorithm** used for encryption/decryption and digital signatures. Its security is based mainly on the difficulty of mathematical problems related to **factoring a large integer nn** into its two prime factors p and q.

Security Aspects of RSA

1. Brute-Force Attack

In a brute-force attack, the attacker tries **all possible private keys** until the correct key is found.

Protection: Use a sufficiently large key space. A larger RSA key makes exhaustive search computationally impractical.

---

2. Mathematical Attacks

Mathematical attacks attempt to determine the private key by exploiting the mathematical structure of RSA.

The main approaches are:

1. **Factor n into p and q**
   Since
   $$n = pq$$
   obtaining p and q allows the attacker to calculate:
   $$\phi(n) = (p-1)(q-1)$$
   and then determine the private key d.
2. **Determine $\phi(n)$ directly** without first finding p and q.
3. **Determine d directly** without first determining $\phi(n)$. BCS703-module-2-textbook

The security of RSA is therefore closely related to the **integer factorization problem**. For a sufficiently large n with large prime factors, factoring is computationally difficult. BCS703-module-2-textbook

---

3. Timing Attack

A timing attack uses the **running time of the RSA decryption operation** to obtain information about the private key.

The attacker collects timing measurements from RSA operations and attempts to determine information about the secret exponent.

Countermeasures:

- **Constant exponentiation time:** Perform operations in approximately the same time.
- **Random delay:** Introduce random delays to make timing measurements less useful.
- **Blinding:** Randomize the ciphertext before performing the private-key operation.

4. Hardware Fault-Based Attack

In this attack, the attacker attempts to induce **faults in the hardware** during RSA operations, particularly during digital-signature generation.

The resulting incorrect signatures or computations can be analyzed to obtain information about the private key.

Such attacks generally require **physical access to the target hardware** and the ability to influence its operation. BCS703-module-2-textbook

---

5. Chosen-Ciphertext Attack

In a chosen-ciphertext attack, the attacker exploits properties of RSA by selecting ciphertexts and studying their corresponding decrypted results.

This type of attack exploits weaknesses in how RSA is implemented or used rather than simply trying to factor n. BCS703-module-2-textbook

---

6. Importance of Key Size

The size of the RSA key has a major effect on security.

- A **small key** makes brute-force and mathematical attacks easier.
- A **large key** increases the computational effort required to attack RSA.
- However, larger keys also increase the computational cost of key generation and encryption/decryption. BCS703-module-2-textbook

Therefore, RSA security requires appropriately large keys and secure implementation.

7. Factoring Problem

The central security assumption of RSA can be represented as:

$$n = p \times q$$

The values p and q are kept secret, while n is public.

If an attacker can efficiently factor n, then they can calculate:

$$\phi(n) = (p-1)(q-1)$$

and subsequently determine:

$$d = e^{-1} \pmod{\phi(n)}$$

Thus, the private key can be compromised. BCS703-module-2-textbook

Summary

The major security threats to RSA are:

Attack	Basic Idea
Brute Force	Try all possible private keys
Mathematical Attack	Exploit RSA mathematics, especially factoring
Timing Attack	Analyze execution time of RSA operations
Hardware Fault Attack	Induce hardware faults and analyze results
Chosen Ciphertext Attack	Exploit properties of RSA using selected ciphertexts

QUESTION 4:

Describe the fundamental requirements that a public key cryptosystem must meet to ensure security.

Answer

A public-key cryptosystem uses **two related keys**:

- **Public key PU** — made publicly available.
- **Private key PR** — kept secret by the owner.

The security of a public-key cryptosystem depends on the fact that knowing the public key should not allow an attacker to determine the corresponding private key.

Fundamental Requirements

A public-key cryptosystem must satisfy the following requirements:

1. Easy Key Generation

It must be **computationally easy** for a user B to generate a pair of related keys:

$$(PU_B, PR_B)$$

where PU_B is the public key and PR_B is the private key.

---

2. Easy Encryption

It must be **computationally easy** for a sender A, knowing the receiver's public key PU_B and the message M, to generate the ciphertext:

$$\boxed{C = E(PU_B, M)}$$

Thus, anyone who has access to the receiver's public key can perform encryption. BCS703-module-2-textbook

---

3. Easy Decryption

It must be **computationally easy** for the receiver B, who possesses the private key PR_B, to recover the original message from the ciphertext:

$$\boxed{M = D(PR_B, C)}$$

Therefore,

$$\boxed{M = D[PR_B, E(PU_B, M)]}$$

The legitimate receiver should be able to perform this operation efficiently. BCS703-module-2-textbook

4. Private Key Must Be Difficult to Derive

It must be **computationally infeasible** for an adversary who knows the public key PU_B to determine the corresponding private key PR_B.

$$\boxed{PU_B \not\Rightarrow PR_B}$$

This is the fundamental security requirement of public-key cryptography. BCS703-module-2-textbook

---

Security Condition

A secure public-key system can be represented as:

```
              PUBLIC KEY
                  PU_B
                    |
                    ↓
Message M → [ ENCRYPTION ] → Ciphertext C
                                  |
                          Insecure Channel
                                  |
                                  ↓
                          [ DECRYPTION ]
                               PR_B
                                  |
                                  ↓
                          Original Message M
```

An attacker may know:

$$PU_B, \quad C, \quad E, \quad D$$

but should not be able to efficiently obtain:

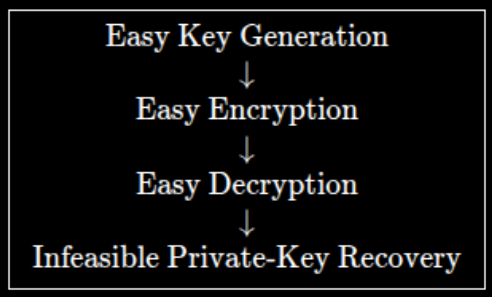
$$PR_B$$

or recover the plaintext from the ciphertext without the appropriate private key. The public-key model specifically requires that knowledge of the algorithm and one key should be insufficient to determine the other key.

Summary

The **four fundamental requirements** are:

1. **Easy key generation** — generate public/private key pair efficiently.
2. **Easy encryption** — encrypt using the recipient's public key efficiently.
3. **Easy decryption** — decrypt using the recipient's private key efficiently.
4. **Computational security** — it must be computationally infeasible to derive the private key from the public key.



Thus, a public-key cryptosystem provides security by making legitimate cryptographic operations efficient while making unauthorized recovery of the private key computationally infeasible. BCS703-module-2-textbook

---

Question 5:

Explain briefly the Elliptic Curve Cryptography (ECC) and mention two applications.

Answer

1. Introduction

Elliptic Curve Cryptography (ECC) is a **public-key cryptographic technique** based on the mathematical properties of elliptic curves.

ECC provides cryptographic functionality similar to RSA and Diffie–Hellman, but it can achieve comparable security with **considerably smaller key sizes**.

2. Elliptic Curve

An elliptic curve over a finite field can be represented in the form:

$$\boxed{y^2 = x^3 + ax + b}$$

where a and b are parameters of the curve.

The points on the curve, together with a special point called the **point at infinity** O, form an elliptic curve group.

ECC uses two important operations:

- **Point addition**
- **Point multiplication**

Point multiplication is repeated point addition:

$$\boxed{kP = P + P + \cdots + P}$$

where P is an elliptic-curve point and k is an integer.

---

3. Principle of ECC

The basic security principle of ECC is the **Elliptic Curve Discrete Logarithm Problem (ECDLP)**.

Suppose:

$$\boxed{Q = kP}$$

It is computationally easy to calculate Q when k and P are known.

However, given P and Q, it is computationally difficult to determine k:

$$\boxed{P, Q \rightarrow k \quad \text{is difficult}}$$

This difficulty provides the basis for ECC security. BCS703-module-2-textbook

---

4. ECC Key Generation

A base point G with a large order n is selected as a public parameter.

Private Key

Each user selects a private integer:

$$\boxed{n_A < n}$$

Public Key

The corresponding public key is calculated as:

$$\boxed{P_A = n_A G}$$

The private key n_A is kept secret, while P_A can be publicly distributed. BCS703-module-2-textbook

---

5. ECC Diffie–Hellman Key Exchange

ECC can be used to perform a key exchange similar to Diffie–Hellman.

Suppose users **A and B** want to establish a common secret.

User A

Private key:

$$n_A$$

Public key:

$$P_A = n_A G$$

User B

Private key:

$$n_B$$

Public key:

$$P_B = n_B G$$

After exchanging public keys:

A calculates:

$$K = n_A P_B$$

B calculates:

$$K = n_B P_A$$

Therefore,

$$n_A P_B = n_A(n_B G)$$

$$= n_B(n_A G)$$

$$\boxed{K = n_B P_A = n_A P_B}$$

Thus, both users obtain the same shared secret. BCS703-module-2-textbook

6. ECC Encryption and Decryption

ECC can also be used for encryption and decryption.

The plaintext message is first represented as an elliptic-curve point:

$$P_m$$

To encrypt the message for user B, sender A chooses a random integer k and generates:

$$\boxed{C_m = \{kG,\ P_m + kP_B\}}$$

where P_B is B's public key.

For decryption, B uses the private key n_B:

$$P_m + kP_B - n_B(kG)$$

Since:

$$P_B = n_B G$$

we get:

$$P_m + k(n_B G) - n_B(kG)$$

$$\boxed{= P_m}$$

Hence, the original plaintext point is recovered. BCS703-module-2-textbook

7. ECC Security

The security of ECC depends on the difficulty of determining k from:

$$Q = kP$$

when P and Q are known.

This is called the **elliptic curve logarithm problem**.

The **Pollard rho method** is a known technique for solving the elliptic curve discrete logarithm problem. ECC achieves strong security with smaller key sizes compared with RSA. BCS703-module-2-textbook

---

8. Advantages of ECC

1. **Smaller key size** for comparable security.
2. Requires **less storage** for keys and certificates.
3. Requires **less bandwidth** for transmission.
4. Suitable for devices with limited computational and storage resources.
5. Provides public-key functionality such as **key exchange and encryption/decryption**.
   BCS703-module-2-textbook

---

9. Applications of ECC

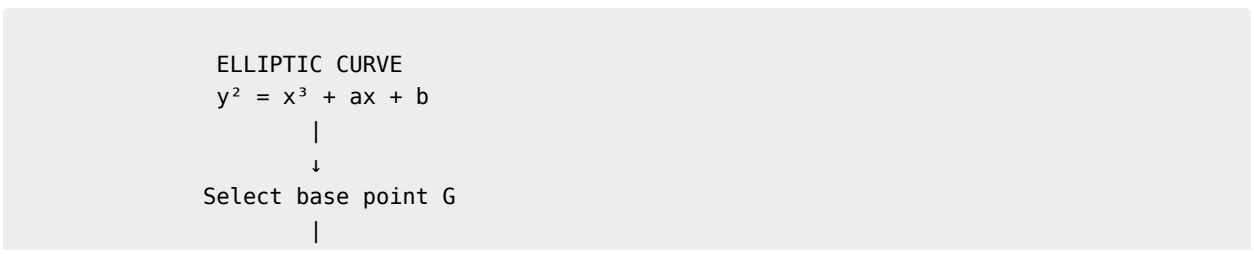
Application 1: Secure Key Exchange

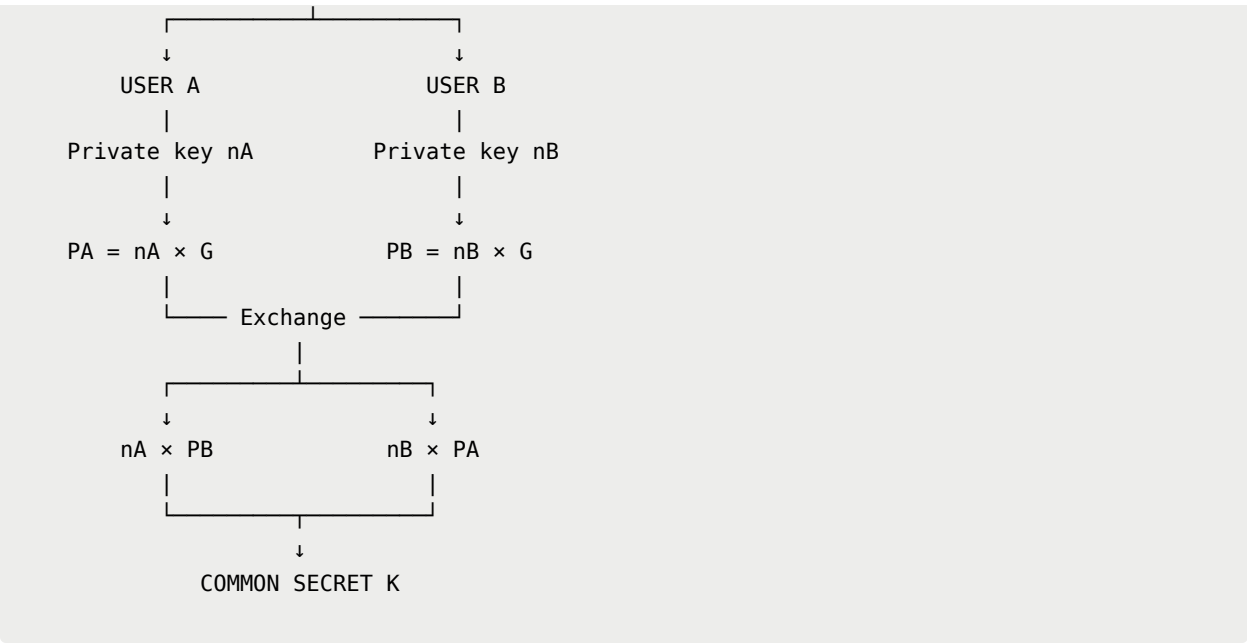
ECC is used for **key exchange**, particularly through elliptic-curve versions of Diffie–Hellman. Two parties can establish a common secret key without directly transmitting the secret. BCS703-module-2-textbook

Application 2: Encryption and Digital Security

ECC can be used in **encryption/decryption systems** and public-key security mechanisms. Its smaller key sizes make it useful where computational resources, storage, or bandwidth are limited.
BCS703-module-2-textbook

10. Simple ECC Diagram





Question 6:

What is the role of a Pseudo Random Number Generator (PRNG) in cryptography?

Answer

Introduction

A **Pseudo Random Number Generator (PRNG)** is an algorithm that generates a sequence of numbers or bits that **appears random** but is actually produced deterministically from an initial value called a **seed**.

In cryptography, random and unpredictable values are essential for generating **keys, initialization values, and other security parameters**.

Role of PRNG in Cryptography

1. Generation of Cryptographic Keys

PRNGs are used to generate random values that can be used as **cryptographic keys**.

A secure PRNG should produce an unpredictable sequence so that an attacker cannot determine the key by observing previously generated values.

2. Generation of Random Seeds

A PRNG requires an initial value called a **seed**.

The security of the generated sequence depends strongly on the secrecy and unpredictability of the seed. Once the seed is known for a deterministic PRNG, the subsequent sequence can generally be reproduced.

3. Generation of Initialization Values

PRNGs can be used to generate values required during cryptographic operations, such as **initialization vectors (IVs)** and other random parameters.

These values help prevent the same plaintext from producing predictable or identical encryption results.

4. Generation of Nonces and Random Parameters

Random values are required in several cryptographic protocols to provide **freshness and unpredictability**.

A PRNG can generate such values for use as nonces, challenges, and other protocol parameters.

5. Improving Cryptographic Security

A cryptographic PRNG should generate a sequence that is **statistically random and computationally unpredictable**.

Ordinary PRNGs may appear random statistically but may not be secure for cryptographic applications. For example, in a linear congruential generator, once the parameters and one generated value are known, subsequent values can potentially be determined.

Therefore, cryptographic applications require a **cryptographically secure pseudorandom bit generator (CSPRBG)**.

Requirements of a Cryptographic PRNG

A cryptographically secure PRNG should have the following properties:

1. **Unpredictability** – An attacker should not be able to predict future output from previously observed output.
2. **Long period** – The generator should not repeat its sequence quickly.
3. **Good statistical properties** – The generated sequence should appear random.
4. **Secure seed** – The seed should be sufficiently unpredictable and protected.
5. **Efficient generation** – Random bits should be generated efficiently.

A CSPRBG is designed so that, given the first kk output bits, there is no feasible algorithm that can predict the next bit with probability significantly greater than 1/21/2.

PRNG Using Cryptographic Algorithms

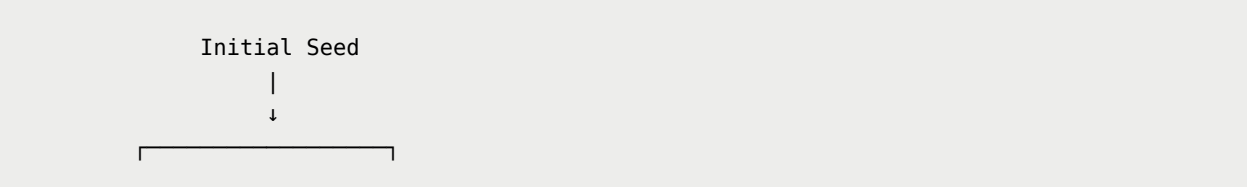
Cryptographic algorithms can themselves be used as the core of PRNGs. Three common approaches are:

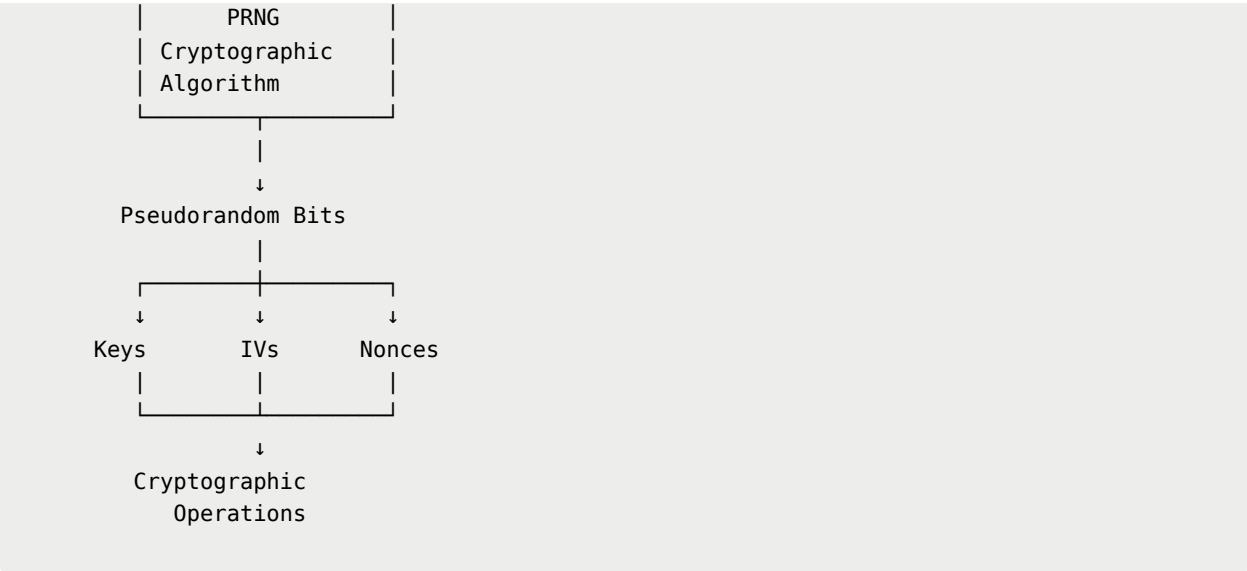
- **Symmetric block ciphers**
- **Asymmetric ciphers**
- **Hash functions and Message Authentication Codes (MACs)**

These approaches can provide cryptographically strong pseudorandom generation.

For example, a symmetric block cipher such as **AES** can be used to construct a PRNG. CTR and OFB modes are established approaches for constructing PRNGs using a block cipher.

Simple Diagram





Question 7:

Write a short note on Public Key Cryptanalysis.

Answer

Introduction

Public key cryptanalysis refers to the analysis and attacks directed against a **public-key cryptosystem** with the objective of recovering the plaintext, private key, or other secret information.

A public-key encryption scheme is vulnerable to several types of attacks. The major attacks are discussed below.

---

1. Brute-Force Attack

In a brute-force attack, the attacker tries possible keys until the correct key is found.

The basic countermeasure is to use a **large key size** so that trying all possible keys becomes computationally impractical.

However, public-key algorithms involve complex mathematical operations, so increasing the key size also increases the computational cost of encryption and

decryption. Therefore, the key size must provide sufficient security while maintaining practical performance.

---

2. Attack on the Mathematical Problem

Another approach is to attempt to determine the **private key from the public key**.

In a public-key cryptosystem, the public key is intentionally available to everyone. Therefore, security requires that it should be computationally infeasible to obtain the private key from the public key.

For example, in RSA:

$$n = pq$$

where p and q are secret prime numbers.

If an attacker can factor n into p and q, the attacker can calculate:

$$\phi(n) = (p-1)(q-1)$$

and subsequently determine the private key.

Thus, the security of RSA is closely related to the difficulty of factoring a large integer.

BCS703-module-2-textbook

The textbook also notes that it has not been mathematically proven that determining the private key from the public key is infeasible for every public-key algorithm. Therefore, the mathematical assumptions underlying a particular algorithm are an important part of its security. BCS703-module-2-textbook

3. Probable-Message Attack

A **probable-message attack** is particularly relevant to public-key encryption.

Suppose an attacker knows that the plaintext belongs to a relatively small set of possible messages. The attacker can:

1. Generate all possible candidate messages.
2. Encrypt each candidate using the victim's public key.
3. Compare the resulting ciphertexts with the intercepted ciphertext.
4. If a match is found, the plaintext can be identified.

Example

Suppose a message consists only of a **56-bit DES key**.

An attacker could encrypt all possible 56-bit DES keys using the public key and compare the resulting ciphertext with the transmitted ciphertext.

Thus, the attack effectively becomes a brute-force search over the smaller message space rather than the public-key key space.

Countermeasure

This attack can be prevented by adding **random bits** to simple or predictable messages before encryption.

Therefore, even if the original message is predictable, the actual input to the public-key encryption algorithm becomes different and unpredictable.

4. General Security Requirement

A secure public-key cryptosystem should make the following task computationally infeasible:

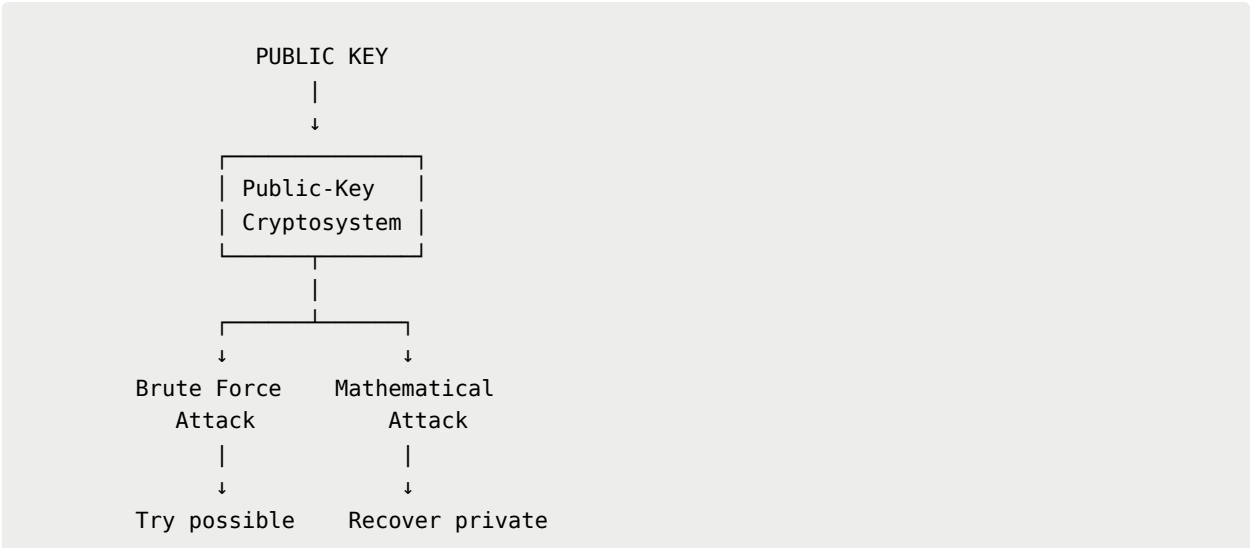
$$\boxed{PU_B \rightarrow PR_B}$$

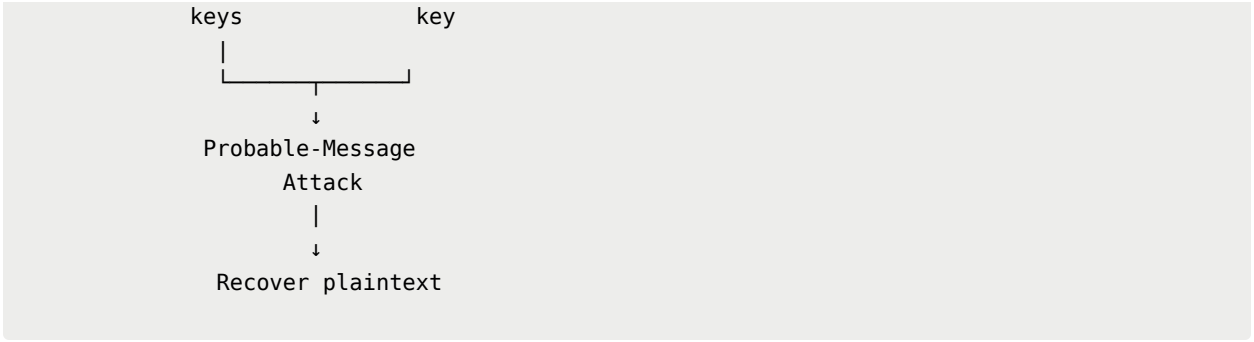
where:

- PU_B = public key
- PR_B = private key

The attacker may know the **encryption algorithm, public key, and ciphertext**, but should not be able to efficiently recover the private key or plaintext. BCS703-module-2-textbook

Diagram





Summary

The important forms of **public key cryptanalysis** are:

Attack	Objective	Countermeasure
Brute-force attack	Try possible keys	Use sufficiently large keys
Mathematical attack	Derive private key from public key	Use secure mathematical parameters and algorithms
Probable-message attack	Guess plaintext from a small message space	Add random bits/padding before encryption